LAB-001

Safe Junos Configuration Workflow

TASK-ONLY WORKBOOK

Assessment tasks without answers

Platform: Juniper vJunos-router 25.4R1.12
Environment: Containerlab
Credential: admin / admin@123

LAB-001 - Task-Only Workbook

Safe Junos Configuration Workflow

Do not open `SOLUTION-GUIDE.md` until the assessment is complete.

Scenario

You are responsible for R1, an SP edge router reachable through both Containerlab management and the in-band `192.168.10.0/24` network. A maintenance request requires configuration changes without losing recoverability or unintentionally overwriting another engineer's work.

Your objective is not merely to make the commands succeed. Demonstrate a controlled workflow consisting of pre-check, candidate review, validation, safe activation, operational verification and rollback readiness.

Completion gate

Mark LAB-001 complete only if you can:

- Repeat the workflow without following a command recipe.
- Explain candidate versus active configuration.
- Recover automatically from a management-breaking change.
- Recover manually with rollback and rescue configuration.
- Explain shared, exclusive and private configuration sessions.
- Produce complete evidence for every milestone.

Rules

- Keep console access available before management fault injection.
- Do not use `commit` until the task explicitly allows it.
- Record `show | compare` before every commit operation.
- Run control-plane and data-plane checks after every committed change.
- Record hypotheses before troubleshooting.
- Do not redeploy the lab to hide a failed recovery attempt.

Task 0 - Deploy and baseline

- Deploy `lab001.clab.yml`.
- Wait until both Junos nodes are healthy.
- Connect to R1 through its Containerlab management address.
- Establish a separate console session to R1.
- Verify all four destinations using `scripts/verify.sh`.
- Record:
- software version;

- system uptime;
- interface state;
- route to each connected subnet;
- current commit history;
- logged-in users.

Evidence

- Deployment output
- Baseline verification output
- R1 operational command output

Task 1 - Candidate versus active configuration

On R1, place the following intended information in the candidate configuration without committing it:

- Building: `JNCIE-LAB`
- Floor: `LAB-001`
- R1-to-R2 interface description: `SP-CORE-LINK`

Prove that:

- The candidate contains the change.
- The active configuration does not yet contain the change.
- Packet forwarding continues to use the active configuration.
- Discarding the candidate removes all pending changes.

Questions

- Which database is modified in configuration mode?
- Does editing a candidate immediately notify routing protocols?
- What is the safest way to discard all uncommitted changes?

Evidence

- Candidate configuration
- Candidate-to-active diff
- Active configuration from operational mode
- Verification before and after discarding the candidate

Task 2 - Validation failure and commit audit trail

Create a policy named `LAB001-TEST` that references a prefix list that does not exist.

- Validate without activating the configuration.

- Capture the complete error.
- Create the missing prefix list containing `192.0.2.0/24`.
- Validate again.
- Review the candidate diff.
- Commit using an audit comment containing the lab ID and change purpose.
- Prove the commit is present in history.

Do not apply the test policy to a routing protocol.

Evidence

- Failed validation
- Corrected validation
- Diff
- Commit history and comment

Task 3 - Automatic rollback

Add rack location `RACK-01`.

- Activate it temporarily with a three-minute confirmation window.
- Prove the value is active.
- Identify the pending rollback deadline.
- Do not confirm it.
- Prove the value disappears after timeout.
- Repeat the change.
- This time, verify it and make it permanent before timeout.

Questions

- Is a confirmed commit active or merely staged?
- What event makes the change permanent?
- What happens when the session closes before confirmation?

Task 4 - Management-loss simulation

Perform this task only with a working console session.

Client1 reaches R1 through `192.168.10.1`. Change R1's client-facing address to `192.168.99.1/24` using a two-minute automatic recovery window.

Requirements:

- Perform and record pre-checks.
- Review the exact diff.

- Validate the candidate.
- Activate it temporarily.
- Prove client1 loses reachability.
- Do not correct the address through console.
- Observe the pending commit through console.
- Prove the original address and reachability return automatically.

Incident note

Record:

- Symptom
- Impact
- Initial hypothesis
- Evidence
- Recovery mechanism
- Post-recovery verification

Task 5 - Manual rollback

Create and commit three distinct rack values, with a unique commit comment for each.

Then:

- Display commit history.
- Load the immediately preceding committed configuration into the candidate.
- Review the reverse diff before activation.
- Commit the rollback with an audit comment.
- Load an older rollback revision and inspect it without activating it.
- Return the candidate to the current active configuration.
- Prove there is no remaining candidate diff.

Questions

- Does loading a rollback immediately change forwarding?
- What does rollback zero represent?
- Why must a rollback be reviewed before commit?

Task 6 - Rescue configuration

- Confirm the router is in a known-good state.
- Save the known-good committed configuration as the rescue configuration.
- Prove the rescue configuration exists.
- Commit an intentionally incorrect rack value.

- Load the rescue configuration into the candidate.
- Inspect the diff.
- Validate and commit the restoration with an audit comment.
- Run the complete verification script.

Questions

- How is rescue different from rollback one?
- When should rescue be refreshed?
- What is the risk of restoring a stale rescue configuration?

Task 7 - Concurrent configuration sessions

Open two SSH sessions to R1.

Shared candidate

- Enter standard configuration mode from both sessions.
- Make a location change in session A without committing.
- Determine whether session B can see it.
- Discard all changes.

Exclusive candidate

- Acquire an exclusive configuration lock in session A.
- Attempt to enter configuration mode from session B.
- Record the result and identify the lock owner.

Private candidates

- Enter private configuration mode from both sessions.
- Modify different location leaves.
- Compare visibility between sessions.
- Commit session A.
- Review and commit session B.
- Repeat using conflicting edits to the same leaf and document the result.

Final assessment - Blind workflow

Without using the solution guide, perform this change request:

> Change the R1-to-R2 interface description to `PROD-CORE-R1-R2`. Maintain service availability, include an audit trail, and demonstrate rollback readiness.

Submit:

- Change plan
- Pre-check output
- Candidate diff
- Validation result
- Activation method and justification
- Post-check output
- Rollback plan
- Final commit history

Scoring

Dimension | Weight

--- | ---:

Candidate/active understanding | 10%

Safe commit workflow | 20%

Confirmed commit recovery | 20%

Manual rollback | 15%

Rescue configuration | 10%

Concurrent sessions | 10%

Operational verification | 10%

Documentation | 5%

Pass requires at least 80% and no critical recovery or verification failure.